

**The non-recursive routine to traverse a binary in inorder is given below:**

```
# define STACKSIZE 100
Struct Stack
{
int top;
NODEPTR item[STACKSIZE];
};
Struct stack S;
Initially S.top=-1;
void inorder (NODEPTR root)
{
NODEPTR p;
p=root;
do
{
while(p!=NULL)
{
push(p);
p=p->left;
}
if(!empty())
{
p=pop();
printf("%d\t", p->info);
p=p->right;
}
}while(!empty() || p!=NULL);
} /* end inorder */
```

## Threaded Binary Trees

If we consider the non-recursive function inorder, we have to use the stack.

The stack is popped when  $p$  becomes NULL. This situation occurred whenever the while loop is exited.

Thus, the top element of the stack is the value of  $p$ , before it became NULL. If a tail pointer  $q$  is kept behind  $p$ , the value of  $q$  can be used directly and need not be popped.

If  $p$  is NULL, then the while loop is exited directly ie, while loop is skipped entirely. This occurs after reaching a node with an empty right subtree, executing the statement,  $p = p \rightarrow \text{right}$  and then returns to repeat the while loop.

Suppose the NULL pointer, in its right field is replaced with a pointer to its inorder successor, then no use of stack.

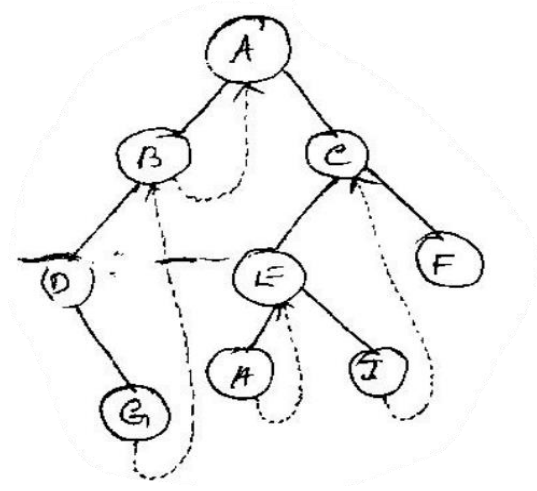
Thread is a pointer, which is connected to a particular node's inorder successor or inorder predecessor.

A right-in threaded binary tree is the one in which the right NULL pointer of a node is given to its inorder successor.

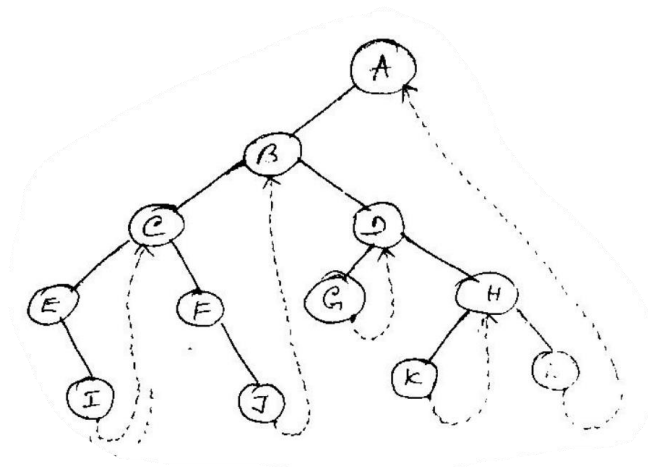
A left-in threaded binary tree is the one in which the left NULL pointer of a node is given to its inorder predecessor.

An in-threaded binary tree is the one in which the left and right NULL pointer of a node is given to its inorder predecessor and inorder successor respectively.

Examples for Right-in threaded binary tree:



fig(a) - Right-in threaded Binary Tree



fig(b) - Right-in threaded Binary Tree

Implementation of Right-in threaded Binary tree :

The implementation of a right-in threaded binary tree under dynamic node implementation, an extra logical field rthread is included within each node to indicate whether or not its right pointer is a thread. ie, the rthread is TRUE if right pointer is NULL or a non NULL thread.

Note: The rthread field of the right most node of a tree (the last node in the inorder traversal) is also set to TRUE, although its right field remains NULL.

Struct node

```
{
    int info;
    Struct node *left;
    Struct node *right;
    int rthread;
};
```

**The function to implement inorder traversal of a right in-threaded binary tree:**

```
void inorder(NODEPTR root)
{
    NODEPTR p, q;
    p=root;
    do
    {
        q=NULL;
        while(p!=NULL)
        {
            q=p;
            p=p→left;
        }
        if(q!=NULL)
        {
            printf("%d", q→info);
            p=q→right;
            while(q→rthread && p!=NULL)
            {
                printf("%d", p→info);
                q=p;
                p=p→right;
            }
        }
    } while(p!=NULL);
}
```

```

} /* end while*
} /* end if */
}while(q!=NULL);
} /* end of function inorder */

```

Creation of Right-in threaded binary tree:

```

NODEPTR maketree(int x)
{
    NODEPTR p;
    p=getnode();
    p→info = x;
    p→left = NULL;
    p→right = NULL;
    prthread= TRUE;
}

```

```

void setleft(NODEPTR p, int x)
{
    NODEPTR q;
    if(p==NULL)
        printf(" Insertion not possible");
    else if(p→left!=NULL)
        printf("Invalid insertion");
    else
    {
        q=getnode();
        q→info=x;
        q→left = NULL;
        p→left = q;
        q→right = p;
    }
}

```

```
q→rthread = TRUE
}
}/* end of function setleft */
```

```
void setright(NODEPTR p, int x)
{
    NODEPTR q;
    if(p==NULL)
        printf(" Insertion not possible");
    else if(!p→rthread)
        printf("Invalid insertion");
    else
    {
        q=getnode();
        q→info=x;
        q→left = NULL;
        temp=p→right; /* Save the inorder successor of node p */
        p→right = q;
        p→rthread = FALSE;
        q→right = temp;
        q→rthread = TRUE
    } /* end of else */
}/* end of function setleft */
```